Network Sentinel: Integrating Machine Learning and Retrieval-Augmented Generation for Network Intrusion Detection and Incident Intelligence

Sharmila Murisetty
Graduate Certificate in Business Intelligence & Systems Infrastructure
Algonquin College

April 13, 2026

Abstract

Network Sentinel is an end-to-end Machine Learning and Retrieval-Augmented Generation (RAG) based intrusion detection system developed for CICIDS-style network traffic analysis. The project addresses a key limitation of conventional intrusion detection workflows: while many systems can classify traffic as benign or malicious, they do not provide contextual explanation, severity interpretation, or analyst-facing guidance. To address this gap, Network Sentinel combines data preprocessing, engineered network flow features, supervised classification, vector-based knowledge retrieval, and structured response generation within a single modular pipeline. The system uses the `Wednesday-workingHours.pcap_ISCX.csv` dataset, originally containing approximately 692,703 rows and 79 columns, which was cleaned to approximately 610,794 rows and expanded to 80 processed features. Three models were compared using an 80/20 stratified train-test split and 5-fold cross-validation: Logistic Regression, Random Forest, and XGBoost. XGBoost emerged as the final deployed model with a cross-validation mean accuracy of 0.9965, a standard deviation of 0.0019, and a test accuracy of 0.9997.

Beyond predictive performance, the system incorporates a ChromaDB-backed cybersecurity knowledge base and an OpenAI or fallback reasoning layer to translate technical outputs into structured incident intelligence. The final application is deployed through a simplified Streamlit dashboard that accepts CSV uploads and returns attack prediction, confidence, severity, summary, and technical details. The resulting architecture moves beyond a basic classification pipeline and instead functions as an analyst-oriented decision support system. This report presents the problem definition, methodology, architecture, workflow, validation results, modular structure, dashboard implementation, logging, error handling, findings, recommendations, and future directions of the project.

Keywords: Network Intrusion Detection, Machine Learning, XGBoost, Retrieval-Augmented Generation, ChromaDB, Streamlit, Cybersecurity Analytics, Decision Support System

1. Introduction

Intrusion detection remains one of the most important applications of data-driven cybersecurity. As enterprise networks generate large volumes of traffic, analysts increasingly require automated systems to distinguish benign patterns from malicious activity. However, many traditional detection pipelines focus narrowly on prediction accuracy and stop at generating alerts. This leaves interpretation, validation, and response planning to human analysts, often under time pressure.

Network Sentinel was developed to address this limitation by combining classification with contextual reasoning. The project integrates Machine Learning (ML) for attack prediction and Retrieval-Augmented Generation (RAG) for attack explanation and mitigation guidance. In this design, the ML model identifies the most likely attack class, while the retrieval layer supplies relevant cybersecurity context from a vector database and the reasoning layer transforms this context into an analyst-friendly summary.

This report presents the technical design and implementation of Network Sentinel as a complete system rather than an isolated model. It covers the problem definition, dataset, preprocessing, feature engineering, validation strategy, model comparison, architecture, workflow, frontend dashboard, logging, error handling, key findings, limitations, and future improvements.

2. Problem Definition

Traditional intrusion detection systems frequently produce alerts without sufficient explanatory depth. In practice, security analysts need more than a predicted class label. They require additional context such as the likely behavior pattern behind the alert, an interpretation of its severity, and a concise explanation of what actions should be considered next.

This creates a gap between *detection* and *operational usability.* A high-performing classifier may identify malicious traffic with excellent accuracy, but without contextual interpretation, its practical usefulness is limited. The problem addressed in this project is therefore twofold:

1. Develop a robust and accurate model for intrusion detection on structured network traffic data.
2. Extend the prediction pipeline into an analyst-facing intelligence workflow by adding contextual retrieval and explanation generation.

Network Sentinel addresses this problem by integrating a supervised classifier with a retrieval and reasoning pipeline, enabling the system to function not only as a prediction engine but also as a lightweight decision support tool.

3. Project Objectives

The objectives of the project are:

- Build a reliable intrusion detection pipeline for CICIDS-style network traffic data.
- Clean and transform raw network data into a model-ready format.
- Compare multiple supervised learning algorithms using quantitative validation.
- Select and deploy the best-performing model based on both accuracy and stability.
- Integrate a ChromaDB-based knowledge layer for contextual retrieval.
- Generate structured incident summaries using an LLM or fallback reasoning agent.
- Present results through a simplified Streamlit dashboard suitable for analyst use.
- Design the solution in a modular manner to improve maintainability, testing, and deployment readiness.

4. Dataset Description

The project uses the file `data/raw/Wednesday-workingHours.pcap_ISCX.csv`, a CICIDS-style network intrusion dataset containing packet flow statistics and traffic behavior features. The raw dataset consisted of approximately 692,703 rows and 79 columns. After cleaning, the dataset was reduced to approximately 610,794 rows. Following preprocessing and feature engineering, the final working feature space consisted of 80 processed features.

The target classes available in the project include:

- BENIGN
- DoS GoldenEye
- DoS Hulk
- DoS Slowhttptest
- DoS slowloris
- Heartbleed

These classes provide a relevant mix of normal traffic and attack behaviors, particularly denial-of-service related traffic patterns and a critical vulnerability-based attack. This makes the dataset appropriate for evaluating classification performance on structured tabular intrusion data.

5. Data Preprocessing and Feature Engineering

5.1 Preprocessing Strategy

The preprocessing pipeline was designed to ensure consistency, numerical stability, and compatibility between training-time and inference-time transformations. This was particularly important because the deployed application accepts uploaded CSV files and must reproduce the same transformation logic used during model training.

The preprocessing steps included:

1. **Column Cleanup:** Column names were stripped of leading and trailing whitespace to prevent schema mismatches.
2. **Duplicate Removal:** Duplicate records were removed to reduce redundancy and bias. Earlier analysis identified 81,909 duplicate rows.
3. **Missing Value Handling:** Missing values in `Flow Bytes/s` were replaced with 0.
4. **Infinite Value Handling:** Positive and negative infinite values were safely replaced to prevent downstream scaler and model failures.
5. **Numeric Coercion:** Feature columns were coerced into numeric format where necessary.
6. **Clipping and Sanity Bounds:** Extreme values were clipped using conservative numeric bounds to avoid overflow or instability.
7. **Final Numeric Sanity Check:** A final validation step ensured that no non-finite values remained before scaling or inference.

This preprocessing strategy ensured that all model inputs were finite, numeric, and aligned with the trained schema.

5.2 Feature Engineering

Two engineered features were added to improve the model's ability to capture traffic intensity and flow magnitude:

- **Total Packets:**

$$\text{Total Packets} = \text{Total Fwd Packets} + \text{Total Backward Packets} \tag{1}$$

- **Packets per Second:**

$$\text{Packets per Second} = \frac{\text{Total Packets}}{\text{Flow Duration} + 1} \tag{2}$$

These engineered features were intended to capture aggregate flow volume and normalized packet intensity, both of which are informative for identifying abnormal traffic behavior in denial-of-service patterns.

5.3 Encoding and Scaling

The target variable `Label` was encoded using a `LabelEncoder`. Input features were normalized using `StandardScaler`. This ensured that the model training process operated on standardized numeric inputs and that deployed inference used the same fitted transformation. Persisting both the label encoder and scaler as artifacts was essential for inference consistency.

6. System Architecture and Modular Design

6.1 System Architecture

Network Sentinel follows a layered, modular architecture designed to separate data handling, prediction, retrieval, reasoning, and presentation. Rather than embedding all logic into a single file, the project distributes responsibilities across specialized modules.

The architecture consists of the following layers:

- **Presentation Layer:** Streamlit dashboard for file upload, analysis trigger, and result display.
- **Application/Orchestration Layer:** `app.py` coordinates the end-to-end sequence.
- **Data Processing Layer:** Cleaning, feature engineering, scaling support, and inference preparation.
- **Machine Learning Layer:** Model training, validation, artifact saving, and deployed inference.
- **Intelligence/Retrieval Layer:** ChromaDB-based knowledge retrieval.
- **Reasoning Layer:** OpenAI generation with fallback response support.
- **Persistence/Support Layer:** Saved artifacts, logs, and vector database.

6.2 Folder Structure

```
Network-Sentinel/
|-- app.py
|-- requirements.txt
|-- README.md
|-- data/
|   |-- raw/
|   |   |-- Wednesday-workingHours.pcap_ISCX.csv
|   |-- processed/
|-- logs/
|   |-- network_sentinel.log
|-- models/
|   |-- model.pkl
|   |-- scaler.pkl
|   |-- label_encoder.pkl
|   |-- feature_columns.pkl
|-- rag/
|   |-- knowledge_base.py
|   |-- retriever.py
|   |-- chroma_db/
|-- src/
|   |-- agent/
|   |   |-- incident_agent.py
|   |   |-- llm_agent.py
|   |-- data/
|   |   |-- loader.py
|   |   |-- preprocessing.py
|   |-- models/
|   |   |-- train.py
|   |   |-- predict.py
|   |-- utils/
|       |-- logger.py
```

6.3 Modularization and File Roles

Table 1 summarizes the role of each major file.

Modularization improves separation of concerns and makes the project easier to extend. For example, the deployed UI can change without modifying the model training logic, and the knowledge base can be expanded independently of the classifier. This structure also supports better testing, debugging, and long-term maintainability.

6.4 Architecture Diagram Description

The architecture can be described conceptually as a directional flow. Figure 1 illustrates the

Table 1: Key Project Modules and Their Roles

File	Role
`app.py`	Streamlit entry point; orchestrates upload, prediction, retrieval, and display
`src/data/loader.py`	Dataset loading logic
`src/data/preprocessing.py`	Cleaning, feature engineering, scaling support, and inference preparation
`src/models/train.py`	Model training, validation, comparison, and artifact saving
`src/models/predict.py`	Artifact loading and deployed inference
`src/agent/incident_agent.py`	Bridges prediction output with retrieval and reasoning
`src/agent/llm_agent.py`	OpenAI-based response generation with fallback support
`rag/knowledge_base.py`	Initializes and seeds the ChromaDB knowledge base
`rag/retriever.py`	Retrieves relevant cybersecurity knowledge from ChromaDB
`src/utils/logger.py`	Centralized logger creation and log management

end-to-end pipeline of the system.

This architecture represents a hybrid ML + RAG system. The Machine Learning component provides highly accurate detection on structured tabular data, while the Retrieval-Augmented Generation layer enriches predictions with contextual cybersecurity knowledge. The reasoning layer transforms technical outputs into analyst-readable insights, and the Streamlit interface presents results in an accessible and operationally useful format.

7. System Workflow

7.1 Step-by-Step Workflow

The end-to-end workflow is:

1. User uploads a network traffic CSV file in Streamlit.
2. The data is loaded into a Pandas DataFrame.
3. Preprocessing and feature engineering are applied.
4. Saved artifacts are loaded from the `/models` directory.
5. XGBoost predicts the most likely attack label.
6. Confidence score is computed from model probabilities.
7. A retrieval query is formed from the prediction.
8. ChromaDB returns relevant cybersecurity context.
9. The LLM or fallback agent generates a structured summary.
10. Results are displayed in the dashboard.

7.2 Analytical Workflow Explanation

The workflow is effective because each stage solves a distinct problem. Preprocessing ensures clean and numerically valid model input. Prediction provides the primary classification signal. Retrieval enriches the output with domain-specific context that a pure classifier does not contain. Generation converts that context into a concise explanation for the analyst. Finally, the dashboard surfaces this intelligence in a readable and operationally useful form.

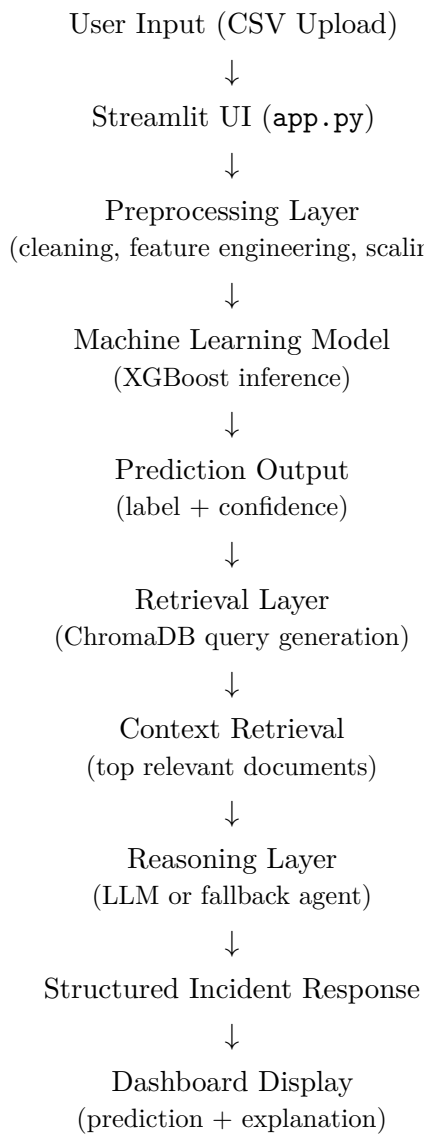

Figure 1: Network Sentinel System Architecture Workflow

7.3 Failure Handling and Robustness

The system includes explicit failure handling rather than failing silently:

- **Missing model artifacts:** Logged error and runtime exception prevent invalid inference.
- **Invalid or empty input data:** Input validation triggers controlled failure messaging.
- **Empty retrieval results:** A default knowledge message is returned instead of crashing.
- **OpenAI API failure:** The fallback agent generates a structured response using retrieved context.

These behaviors improve reliability, especially during deployment and demonstrations. Logging also makes the failure path traceable.

7.4 Why the Workflow Matters

Traditional ML pipelines end with a prediction. Network Sentinel extends this into retrieval, reasoning, and presentation. This transforms a prediction system into a decision support system.

In other words, the workflow bridges the gap between detection and interpretation, enhances analyst usability, and improves operational relevance in realistic cybersecurity workflows.

7.5 Traditional Intrusion Detection Systems vs Network Sentinel

Traditional intrusion detection systems generate alerts only, often with limited context and no reasoning support. In contrast, Network Sentinel provides prediction, confidence, severity, contextual explanation, and mitigation guidance. This differentiation positions Network Sentinel as an intelligent intrusion analysis system rather than a conventional alerting tool.

8. Model Development and Validation

8.1 Training Setup

Model development used an 80/20 train-test split with stratification to preserve class distribution. To improve robustness, 5-fold cross-validation was applied during model comparison. This validation setup ensured that model selection was not based solely on a single holdout result.

Three models were evaluated:

1. Logistic Regression
2. Random Forest
3. XGBoost

This comparison provided both a linear baseline and two tree-based methods more suitable for structured intrusion data.

8.2 Validation Metrics

The primary comparison metrics used in the project were:

- Cross-validation mean accuracy
- Cross-validation standard deviation
- Test set accuracy

These metrics were selected to capture both performance and stability. Test accuracy alone can appear strong even when a model is less stable across folds, while cross-validation standard deviation helps quantify reliability.

9. Model Comparison and Final Selection

Table 2: Model Comparison Results

Model	CV Mean	CV Std	Test Acc.
Logistic Regression	0.9759	0.0241	0.9919
Random Forest	0.9938	0.0054	0.9995
XGBoost	0.9965	0.0019	0.9997

The results show that tree-based methods outperformed Logistic Regression. Random Forest performed strongly, but XGBoost achieved the best overall balance of accuracy and stability. Specifically, XGBoost had the highest test accuracy (0.9997), the highest cross-validation mean (0.9965), and the lowest cross-validation standard deviation (0.0019).

XGBoost was therefore selected as the final deployed model. This result is consistent with expectations for structured tabular intrusion data, where boosting methods often capture nonlinear relationships and complex feature interactions more effectively than linear models.

10. Retrieval-Augmented Generation Layer

The RAG component extends the classifier into an intelligence pipeline. A ChromaDB vector database stores cybersecurity knowledge such as attack descriptions, indicators, impact context, and mitigation guidance. The collection used in the project is `network_security`.

When an attack label is predicted, the system dynamically constructs a retrieval query from that label. ChromaDB then returns the most relevant document or context snippet. This context is passed to the reasoning layer, where an OpenAI-powered generator or fallback logic produces a structured analyst-facing summary.

This design is important because even a very accurate model does not explain *why* the detected pattern matters or *what* action should follow. The RAG layer solves this limitation by augmenting the predicted result with actionable context.

11. Dashboard / Frontend Implementation

The frontend is implemented using Streamlit and was intentionally simplified to reduce clutter. The interface supports the following actions:

- Upload a CSV file
- Run the analysis pipeline
- View predicted attack
- View confidence score
- View severity
- View number of rows processed
- Read incident summary
- Expand technical details

This simplified UI design was chosen deliberately. Rather than overloading the interface with multiple panels and repeated content, the dashboard emphasizes readability and directness. This makes the system more suitable for demonstration and easier for a non-developer stakeholder to understand.

12. Saved Artifacts

The trained pipeline saves four critical artifacts in the `/models` directory.

Table 3: Saved Model Artifacts

Artifact	Purpose
`model.pkl`	Trained XGBoost model used for inference
`scaler.pkl`	Saved `StandardScaler` for feature transformation
`label_encoder.pkl`	Converts predicted numeric labels back to attack names
`feature_columns.pkl`	Preserves the ordered feature schema for inference consistency

Persisting these files ensures that deployment-time inference uses exactly the same transformation and schema assumptions as training.

13. Logging and Error Handling

Runtime events are logged to:

`logs/network_sentinel.log`

The logging system records artifact loading, preprocessing, feature transformation, prediction, retrieval, fallback behavior, and exception handling. This supports several important goals:

- **Debugging:** Errors can be traced to the exact stage of failure.
- **Transparency:** The operational flow becomes auditable.
- **Reproducibility:** Runtime behavior can be reviewed across tests and deployments.
- **Deployment Validation:** Cloud deployment issues become easier to diagnose.

Error handling was implemented using controlled exceptions, validation checks, and fallback logic. In particular, the OpenAI dependency was not allowed to become a single point of failure. If the API fails due to quota or network issues, the fallback agent still returns a structured response using retrieved context.

14. Findings and Key Results

Several findings emerged from the project:

1. Tree-based models clearly outperformed Logistic Regression on this structured intrusion dataset.
2. XGBoost delivered the strongest combination of test accuracy and cross-validation stability.
3. The modular architecture improved maintainability and simplified integration between components.
4. The RAG layer significantly strengthened the system by providing context and mitigation guidance rather than raw prediction alone.
5. The fallback strategy improved robustness and made the application more reliable during demonstrations and deployment.

The most important overall result is that the project successfully moved beyond classification and toward contextual analyst assistance.

15. Recommendations

Based on the current implementation, the following recommendations are appropriate:

- Add full batch attack distribution analysis instead of summarizing only the uploaded input as a whole.
- Include additional evaluation metrics such as precision, recall, F1-score, and confusion matrix.
- Expand the ChromaDB knowledge base with richer cybersecurity content.
- Support optional analyst follow-up queries in an advanced mode.
- Add downloadable incident reports for operational use.
- Strengthen deployment dependency management and environment pinning.
- Extend dataset coverage to additional attack classes for broader generalization.

These changes would increase both the academic strength and practical utility of the project.

16. Limitations

The project has several limitations:

- The current app summarizes the uploaded input rather than performing a full row-level or batch distribution analysis.
- The RAG knowledge base is curated and limited in size.
- LLM output depends on API availability, although the fallback agent reduces this dependency risk.
- Model evaluation currently emphasizes accuracy and cross-validation stability more than a full classification metric suite.

These limitations do not undermine the core contribution of the project, but they identify clear opportunities for future strengthening.

17. Future Improvements

Future work can expand the project in several directions:

- Add batch-level attack distribution visualizations

- Highlight top suspicious rows within uploaded files
- Include confusion matrix and full classification report
- Enrich the knowledge base with broader attack coverage
- Add analyst follow-up query support
- Enable downloadable PDF or JSON incident reports
- Improve secret handling and deployment environment management

These improvements would make Network Sentinel more mature as both an academic project and a deployable applied analytics system.

18. Conclusion

Network Sentinel demonstrates how Machine Learning can be combined with retrieval and reasoning to create a more useful intrusion detection workflow. Using the `Wednesday-workingHours.pcap_ISCX.csv` dataset, the project developed a robust preprocessing pipeline, engineered meaningful traffic features, compared three candidate models, and selected XGBoost as the final deployed classifier based on strong quantitative evidence. With a cross-validation mean accuracy of 0.9965, cross-validation standard deviation of 0.0019, and test accuracy of 0.9997, the model achieved excellent predictive performance.

More importantly, the system did not stop at classification. By integrating ChromaDB and an LLM/fallback layer, the project transformed raw predictions into structured incident intelligence. This hybrid architecture bridges the gap between detection and interpretation, enhances analyst usability, and improves operational relevance. As a result, Network Sentinel should be understood not merely as a machine learning classifier, but as a modular, explainable, and deployment-oriented decision support system for network intrusion analysis.

19. bibliography

T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

Streamlit, "Streamlit Documentation," Available: https://docs.streamlit.io/

Chroma, "ChromaDB Documentation," Available: https://docs.trychroma.com/

OpenAI, "OpenAI API Documentation," Available: https://platform.openai.com/docs/